ITA06 - Machine Learning

Exp 7:

Write a program to implement Logistic Regression (LR) algorithm in python

Aim : To implement the Logistic Regression (LR) algorithm using Python and evaluate its performance using the Confusion Matrix and Accuracy Score.

Program:

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
# ------------------------------------------------------
# Step 1 : Create Synthetic Dataset
# ------------------------------------------------------
np.random.seed(42)
# Class 0
class0 = np.random.multivariate_normal(
    mean=[0, 0],
    cov=[[1, 0.5], [0.5, 1]],
    size=50
)
# Class 1
class1 = np.random.multivariate_normal(
    mean=[3, 3],
    cov=[[1, 0.5], [0.5, 1]],
```

```
    size=50
)

# Combine data
X = np.vstack((class0, class1))
y = np.hstack((np.zeros(50), np.ones(50)))

# Create DataFrame
df = pd.DataFrame(X, columns=["Feature_1", "Feature_2"])
df["Target"] = y.astype(int)
# ----------------------------------------------------
# Step 2 : Display Dataset
# ----------------------------------------------------
print("========== SAMPLE DATASET ==========\n")
print(df.head())
print("\n========== CLASS DISTRIBUTION ==========")
print(df["Target"].value_counts())
# ----------------------------------------------------
# Step 3 : Scatter Plot
# ----------------------------------------------------
plt.figure(figsize=(8,6))
sns.scatterplot(
    x="Feature_1",
    y="Feature_2",
    hue="Target",
    data=df,
    palette="viridis",
    s=100
```

```
)
plt.title("Synthetic Dataset")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid(True)
plt.show()
# ----------------------------------------------------
# Step 4 : Split Dataset
# ----------------------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42
)
print("\nTraining Samples :", len(X_train))
print("Testing Samples  :", len(X_test))
# ----------------------------------------------------
# Step 5 : Train Logistic Regression Model
# ----------------------------------------------------
model = LogisticRegression()
model.fit(X_train, y_train)
# ----------------------------------------------------
# Step 6 : Prediction
# ----------------------------------------------------
y_pred = model.predict(X_tes
# ----------------------------------------------------
# Step 7 : Display Prediction
```

```
# ------------------------------------------------------
result = pd.DataFrame({
    "Actual": y_test.astype(int),
    "Predicted": y_pred.astype(int)
})
print("\n========== ACTUAL vs PREDICTED ==========\n")
print(result)
# ------------------------------------------------------
# Step 8 : Confusion Matrix
# ------------------------------------------------------
cm = confusion_matrix(y_test, y_pred)
print("\n========== CONFUSION MATRIX ==========")
print(cm)
# Heatmap
plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Class 0','Class 1'],
    yticklabels=['Class 0','Class 1']
)
plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.show()
# ------------------------------------------------------
```

```
# Step 9 : Accuracy
# ------------------------------------------------------
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy = {:.2f}%".format(accuracy*100))
# ------------------------------------------------------
# Step 10 : Classification Report
# ------------------------------------------------------
print("\n========== CLASSIFICATION REPORT ==========\n")
print(classification_report(y_test, y_pred))
# ------------------------------------------------------
# Step 11 : Actual vs Predicted Graph
# ------------------------------------------------------
plt.figure(figsize=(10,5))
plt.plot(
    y_test,
    'bo-',
    linewidth=2,
    markersize=7,
    label='Actual'
)
plt.plot(
    y_pred,
    'r*-',
    linewidth=2,
    markersize=8,
    label='Predicted'
)
plt.title("Actual vs Predicted Labels")
```

```
plt.xlabel("Sample Index")
plt.ylabel("Class Label")
plt.legend()
plt.grid(True)
plt.show()
# ----------------------------------------------------
# Step 12 : Accuracy Bar Graph
# ----------------------------------------------------
plt.figure(figsize=(6,5))
plt.bar(
   ["Logistic Regression"],
   [accuracy*100],
   color="green"
)
plt.ylabel("Accuracy (%)")
plt.title("Model Accuracy")
plt.ylim(0,100)
for i, v in enumerate([accuracy*100]):
   plt.text(i, v+1, f"{v:.2f}%", ha='center', fontsize=12)
plt.show()
# ----------------------------------------------------
# Step 13 : Decision Boundary
# ----------------------------------------------------
x_min, x_max = X[:,0].min()-1, X[:,0].max()+1
y_min, y_max = X[:,1].min()-1, X[:,1].max()+1
xx, yy = np.meshgrid(
   np.arange(x_min, x_max, 0.02),
   np.arange(y_min, y_max, 0.02)
```

```
)
Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.figure(figsize=(8,6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='viridis')
sns.scatterplot(
    x=X[:,0],
    y=X[:,1],
    hue=y.astype(int),
    palette='viridis',
    s=80
)
plt.title("Logistic Regression Decision Boundary")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid(True)
plt.show()
```

```
========== SAMPLE DATASET ==========
```

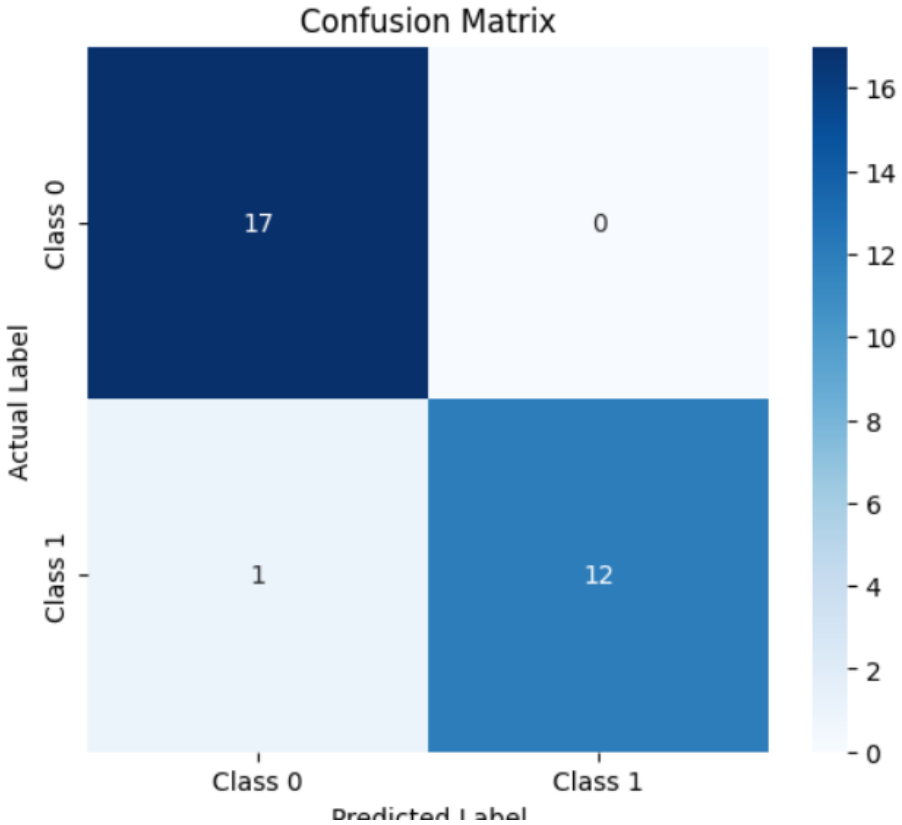
I output actions

```
   ture_1  Feature_2  Target
0 -0.361035 -0.499299       0
1 -1.322430  0.200600       0
2  0.319851  0.085714       0
3 -1.751356 -0.983921       0
4  0.135297  0.677857       0

========== CLASS DISTRIBUTION ==========
Target
0    50
1    50
Name: count, dtype: int64
```

```
    Actual  Predicted
0        1          1
1        1          0
2        1          1
3        0          0
4        0          0
5        0          0
6        0          0
7        1          1
8        0          0
9        0          0
10       0          0
11       0          0
12       1          1
13       0          0
14       1          1
15       0          0
16       1          1
17       1          1
18       0          0
19       0          0
20       1          1
21       1          1
22       0          0
23       0          0
24       1          1
25       0          0
26       0          0
27       1          1
28       0          0
29       1          1
```

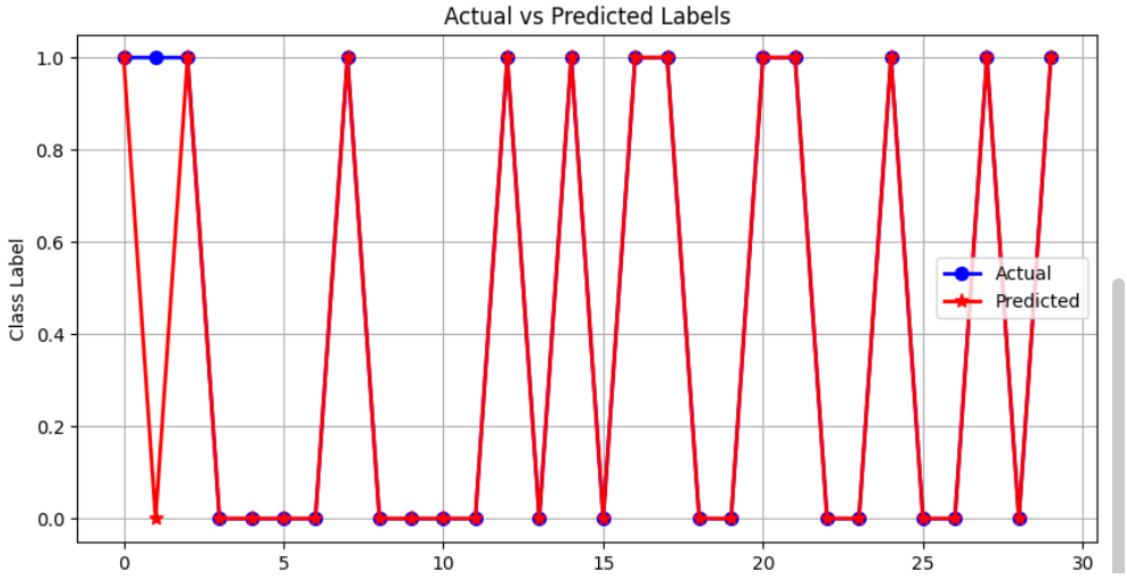
```
========== CONFUSION MATRIX ==========
[[17  0]
 [ 1 12]]
```

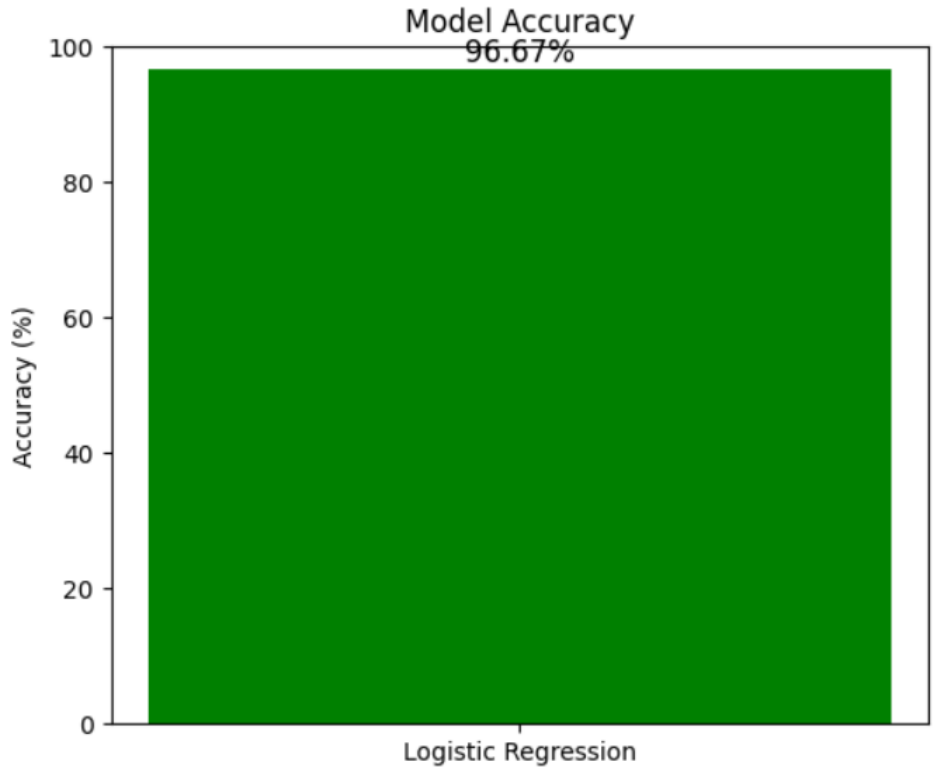


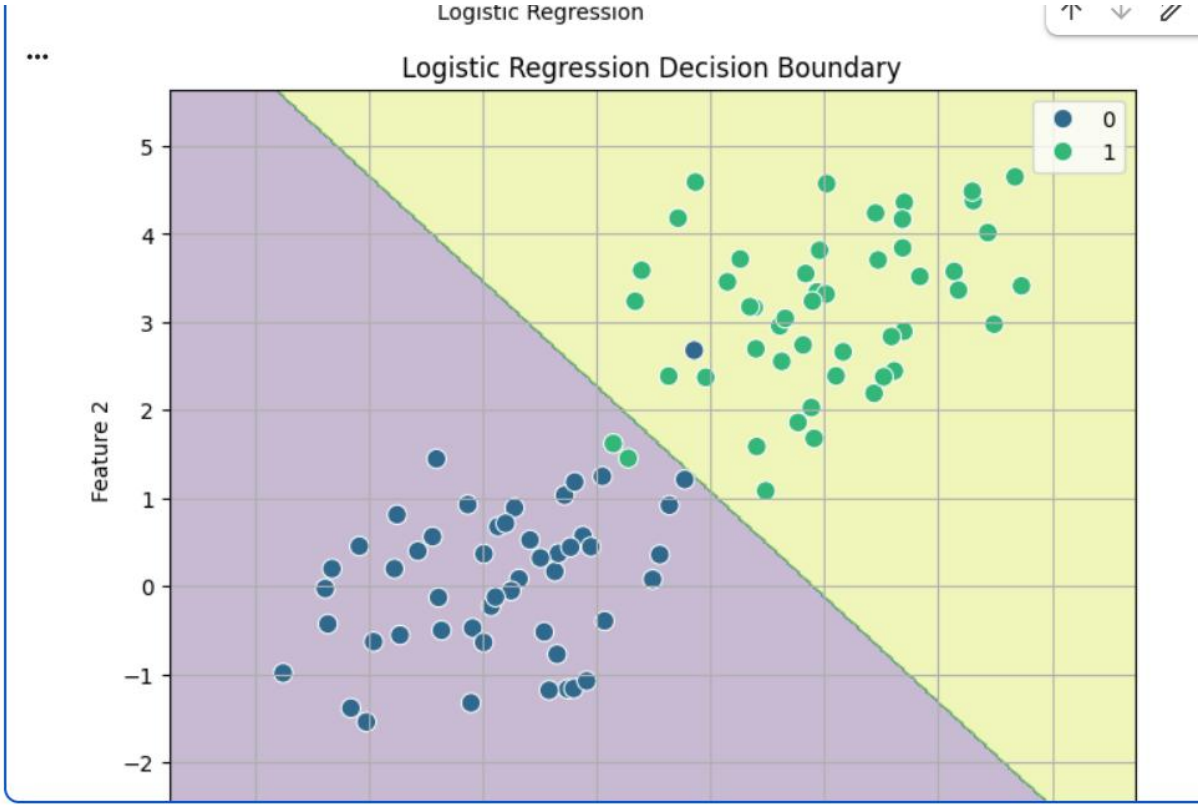

```
              precision    recall  f1-score   support

         0.0       0.94      1.00      0.97        17
         1.0       1.00      0.92      0.96        13

    accuracy                           0.97        30
   macro avg       0.97      0.96      0.97        30
weighted avg       0.97      0.97      0.97        30
```




Model Accuracy
96.67%
Accuracy (%)
100
80
60
40
20
0
Logistic Regression


Logistic Regression
Logistic Regression Decision Boundary
Feature 2
0
1
5
4
3
2
1
0
−1
−2